SYSTEM DESIGN

Event-Driven Architecture Visual Guide

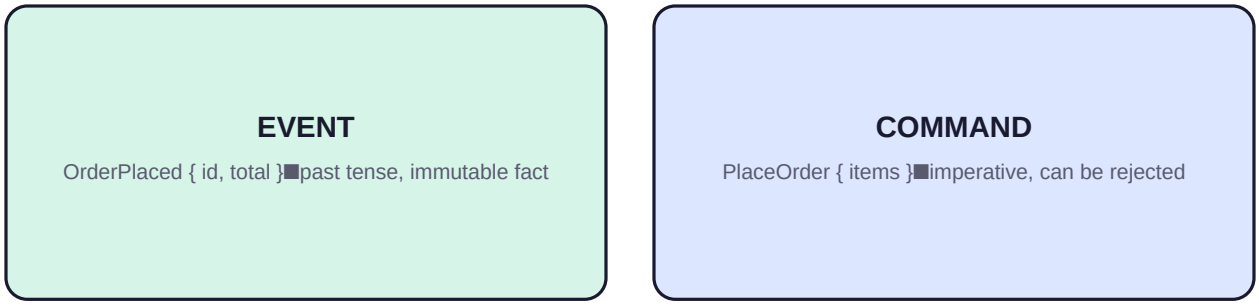
Systems that react to facts:
events, choreography, sourcing, and CQRS.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

Events vs Commands

A fact about the past vs a request for the future -- never confuse them.

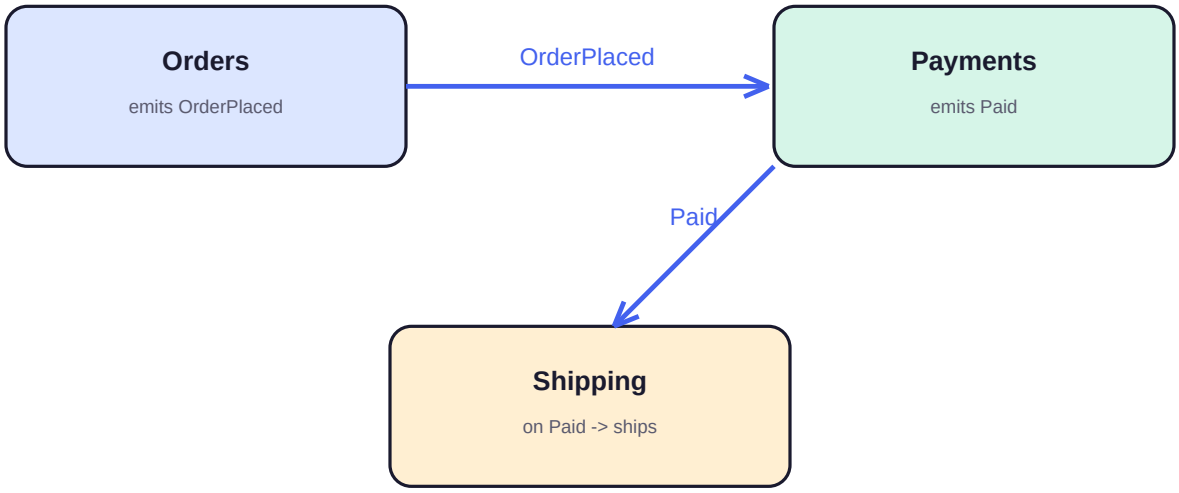


- Events are broadcast: zero or many listeners. Commands target ONE handler that may say no.
- Naming rule: events in past tense (PaymentCaptured), commands as verbs (CapturePayment).
- Event-carried state: include enough payload that listeners skip a callback query.

STEP 2/6

Choreography: No Conductor

Each service reacts to events and emits its own -- the flow emerges.

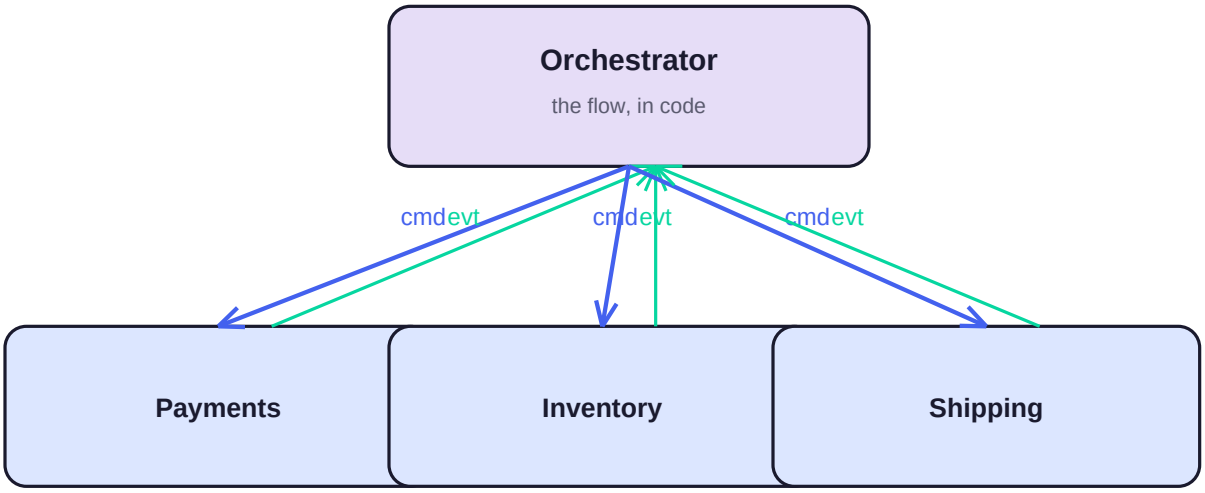


- Pros: maximally decoupled -- add a Fraud service by subscribing, touching nothing.
- Cons: the business flow lives NOWHERE -- debugging means tracing events across services.
- Rule of thumb: choreography for simple flows (3 or fewer steps); orchestrate beyond that.

STEP 3/6

Orchestration: A Conductor

One saga orchestrator tells each service what to do next.

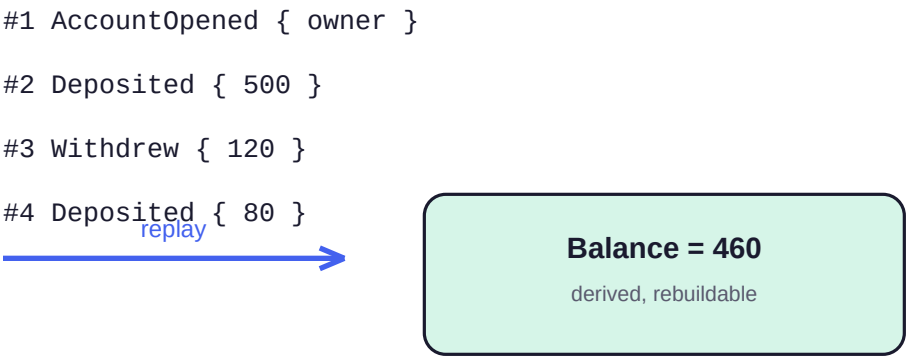


The flow is explicit and testable -- but the orchestrator is a coupling point and a scaling bottleneck.

STEP 4/6

Event Sourcing

Store the events, not the current state. State is a left-fold over history.

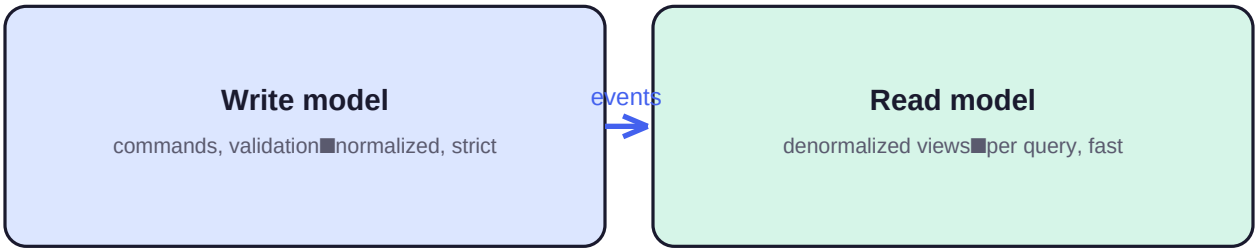


- Audit trail for free: every state change is recorded with cause and time.
- Time travel: rebuild state as of any event. New projections from old events.
- Costs: event schema evolution, ever-growing log (snapshots bound replay), unfamiliar modeling.

STEP 5/6

CQRS: Split Reads from Writes

One model to change the world, another to query it.



- Write side enforces invariants; read side is a materialized view built from its events.
- Reads and writes scale INDEPENDENTLY -- and the read model can be rebuilt from scratch.
- Do NOT start here: CQRS doubles your models. Earn it with real read/write asymmetry.

STEP 6/6

Pitfalls of Event-Driven Systems

The tax you pay for all that decoupling.

Ordering

events can arrive out of order -- design for it (versions, idempotency)

Duplicates

at-least-once delivery -- every handler must be idempotent

Schema evolution

old events live forever -- version payloads, never break readers

Observability

correlate with causation IDs across services, or go blind

STEP ✓/✓

Key Takeaways

Event-Driven Architecture — Remember

- Events are immutable facts (past tense); commands are requests one handler may reject.
- Choreography decouples via reactions; orchestration centralizes the flow -- choreograph simple, orchestrate complex.
- Event sourcing stores history and derives state; CQRS splits write and read models.
- Every event handler must be idempotent and tolerant of out-of-order, duplicated events.
- Version event schemas forever; trace with causation IDs or lose the plot.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev